

Virtual Memory Management: Intro

Dr. Jit Mukherjee

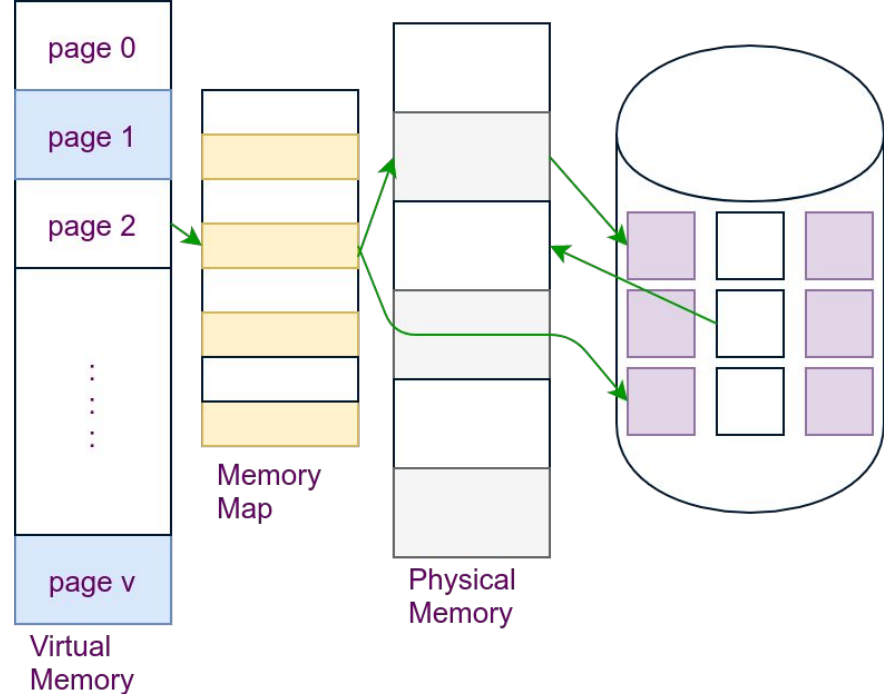


Introduction

- As many process in main memory -> multiprogramming
 - Entire process in main memory to be execute
 - Dynamic loading can ease to a certain bit
- Execution of process that is not in main memory -> program can be larger than physical memory.
 - Abstract main memory to a larger uniform array of storage
 - Share file easily and shared memory
 - Not easy to implement but extremely helpful from user's perspective.
 - User can write program for extremely large virtual address space
 - Degree of multiprogramming
 - Less I/O needed to be loaded or swap
- Not needed parts-
 - Unusual error conditions, certain operations rarely used
 - Array, list, and tables often have more memory than needed.

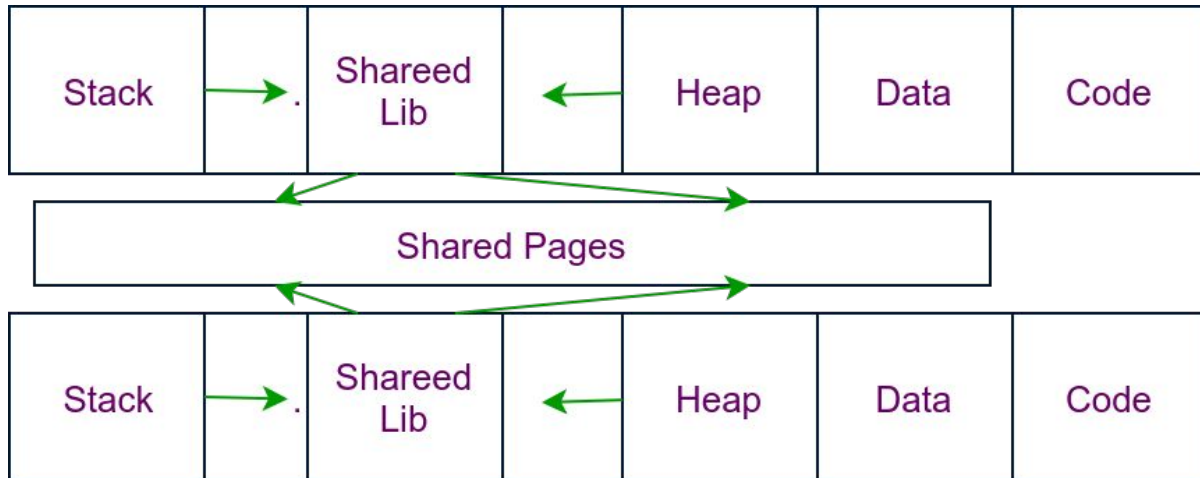
Virtual Memory

- Virtual memory separates logical memory from physical
 - Programmers no need to worry about the available memory
- Virtual address space - virtual view how a process is stored in memory.
 - Non-Contiguous memory, MMU
- Stack -> Downwards, Heap -> Up
 - This gap is part of virtual address space
 - Require actual physical pages if needed
 - Sparse Address Space



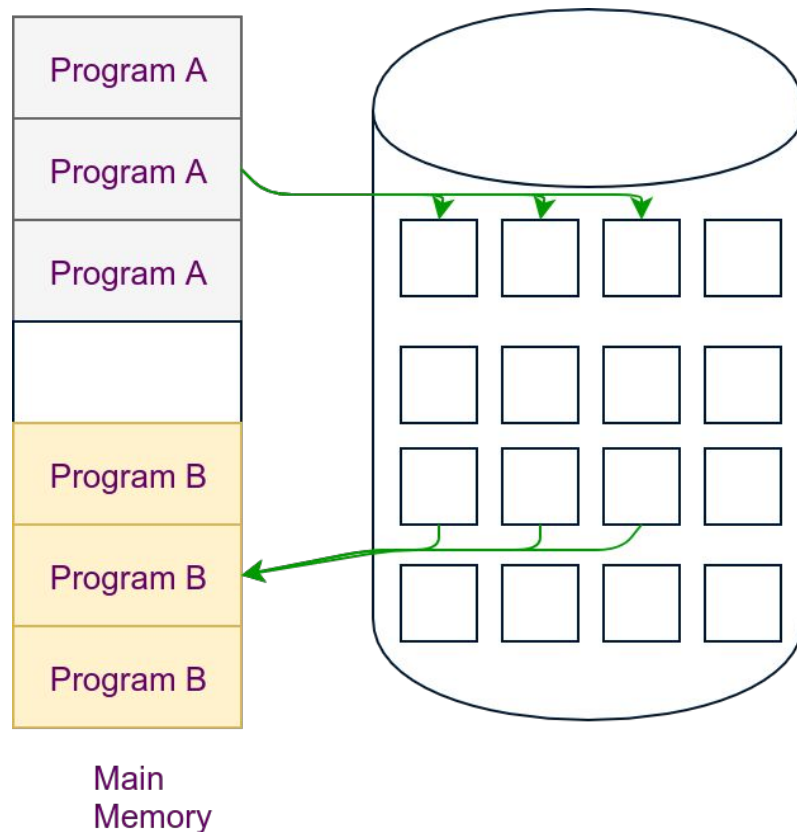
Shared Virtual Memory

- Files and memory to be shared by multiple processes
- Mapping of shared objects into a virtual address space. Read only
- Shared memory IPC
- `fork()` can create a shared memory



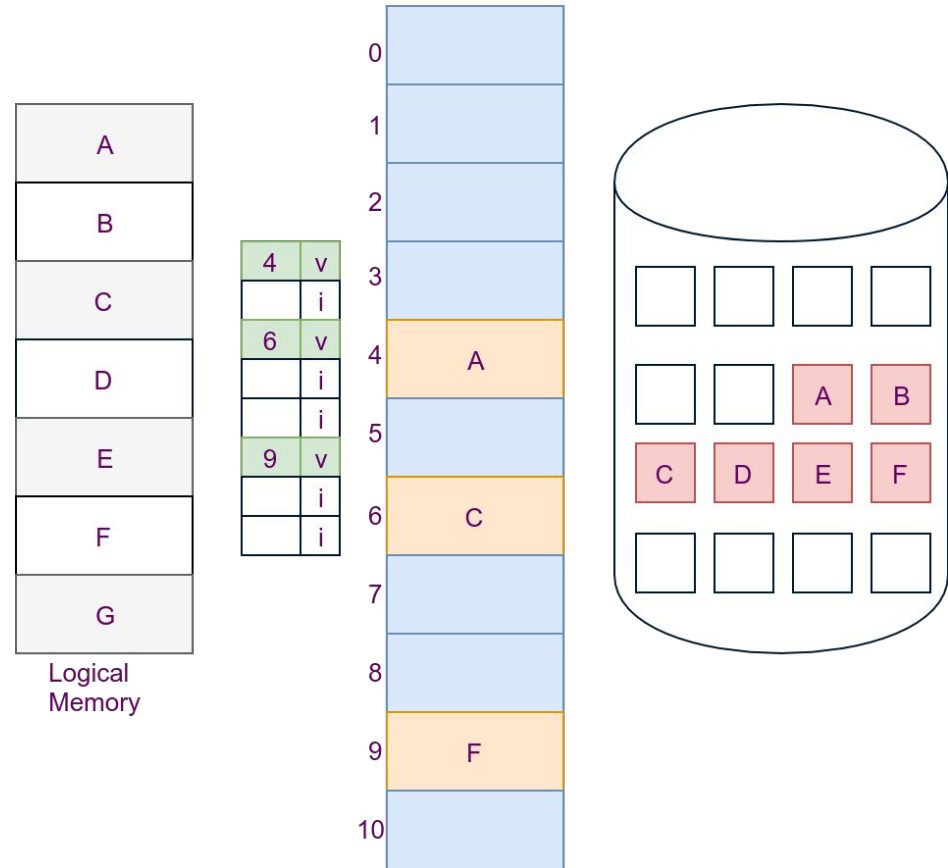
Demand Paging

- Entire program in physical memory at execution time
 - We may not initially need the entire program
- Alternate - load the pages only as they are needed - **Demand Paging**
- Similar to swapping - process in secondary memory -> rather than swapping the entire process, we swap a page only when it is needed -> **Lazy Swapper**
- **Pager**



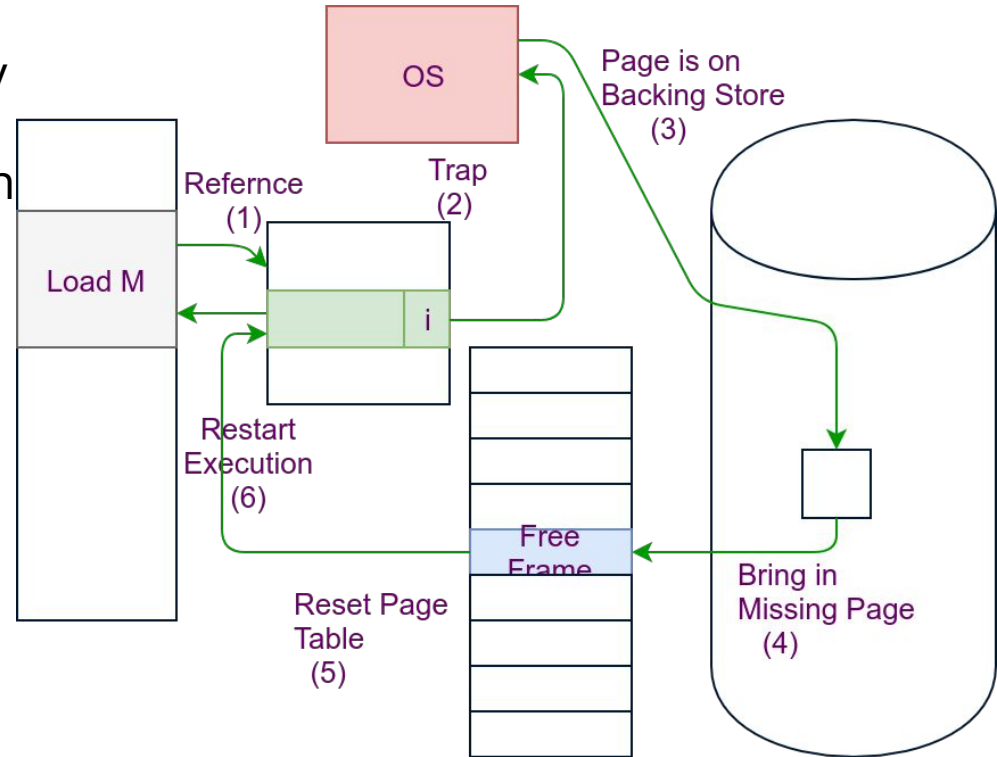
Basic Concepts

- Pager bring only necessary pages to memory.
 - Avoid reading unnecessary memory pages
- Distinguish pages in memory and pages in disk
- Valid-invalid bit
 - Valid - legal and in memory
 - Invalid - illegal or not in memory
 - Memory resident
- **Page-fault trap** - invalid bit
 - Check internal table - memory access
 - Invalid - terminate, Valid
 - Find a frame, read disk operation, modify internal table
 - Restart from the interrupted space



Basic Concepts

- In extreme cases, start executing process with no pages in memory
-> Pure Demand Paging
- Multiple page fault per instruction
-> Performance -> Locality of reference
- Hardware -> Page Table, Secondary Memory (**Swap Space** - faster than file system)
- Instruction fetch -> restart
 - Operand -> fetch and decode
- Fetch and Add
 - Fetch A, B. Add A, B. Store C (fault)
- Several different locations
 - Temporary registers
- Page fault != fatal (demand)



Performance of Demand Paging

- Effective memory access time = $(1-p) * ma + p * \text{page fault time}$
 - ma -> memory access time. p -> probability of a page fault ($0 \leq p \leq 1$)
 - ma - 200 nanosec, pft - 8 milisec
- Service the page-fault interrupt, read in page, resume restart
- $E_{mat} \rightarrow 200 + 7999800 * p$
- **Page fault rate**
- Page fault sequence
 - Trap to the OS
 - Save the user registers and process state
 - Determine if interrupt is a page fault
 - Check page reference is legal and determine the location of the page in the disk
 - Issue a read from the disk to a free frame
 - Wait in a queue for this device until the read request
 - Wait for the device seek and latency time
 - Begin the transfer
 - While waiting allocate CPU to other -> CPU scheduling
 - Receive an interrupt for I/O -> I/O completes
 - Save the state of that process
 - Determine the interrupt from the disk
 - Correct page table and others -> show in memory
 - CPU scheduling
 - Restore and Resume

Copy-on-Write

- `fork()` -> creating a copy of parent's memory space.
- Copy-on-Write -> initially shares same memory -> if either writes to a shared page, a copy is created
- **Pool** of free page from where it will be allocated -> typically stack or heap of a process expands -> **Zero-fill-on-demand** = erase previous element

